

# Building your own text-generating AI (2/4)

*Lecture 9 – Recurrence and the problem of memory*

Dr Thomas Robinson

August 13, 2026

# Warm-up · three questions from last time

---

≈ 5 MIN · SHOUT IT OUT

1. A bigram language model predicts the next token from how much context?
2. What is the cosine similarity of  $[1, 0]$  and  $[0, 1]$ ?
3. An embedding table for 27 tokens, with 8 dimensions per token — how many parameters?

# Warm-up · answers

1. **Exactly one token** — the current one. However long the text, the bigram model conditions on the single most recent token and nothing further back.
2.  $\frac{(1 \times 0) + (0 \times 1)}{1 \times 1} = \frac{0}{1} = 0$ . The vectors are **orthogonal** — no overlap in direction at all.
3.  $27 \times 8 = 216$ . No bias term to add: an embedding table is a lookup table, not a layer computation.



# Part 1 · Why feedforward fails on sequences

# The fixed-input problem

## PICKING UP FROM YESTERDAY

Last time our bigram model remembered exactly one character

- Today we want to give it a running memory

A feed-forward network has a **fixed input size**:

- MLP:  $\mathbf{x} \in \mathbb{R}^d$ , one  $d$ .
- CNN:  $H \times W \times C$  images.

But text is **variable length**. "Yes." has four characters.  
The entirety of *Hamlet* has 175,000.

## Insufficient workarounds

- Pad / truncate to fixed length (*wastes capacity*)
- Bag-of-words / TF-IDF (*throws away order entirely*)
- Sliding window (*caps memory at the window size*)

We want something that **scales with the sequence**.

# Sequence ≠ independent observations

The matters order words in of a sentence the.

This introduces dependencies:

- *The keys, which had been sitting on the table next to the mail and the half-empty coffee mug, **were** still there.*
  - Notice that “were” corresponds to a subject 16 words away
- *He said he would come.* vs *He said he would **not** come.*
  - Single tokens can flip meaning.
- *Hamilton, the founder of the Federalist Party, ...*
  - A co-reference to a named individual several sentences earlier

A good model has to **carry information forward** as it reads.

# Sequences are everywhere

NOT JUST TEXT

## CS / industry

- **Speech recognition** — waveform → words.
- **Music generation** — sequences of MIDI events.
- **Time-series forecasting** — stock prices, server load.
- **DNA / protein sequences** — amino-acid order matters.
- **Streaming / online learning** — events as they arrive.

## Social science

- **Conflict event sequences** — Hegre et al. forecasting.
- **Legislative debate transcripts** — order of speakers, framing.
- **Roll-call voting** — sequence of votes within a session.
- **Survey responses over time** — panel data with attrition.
- **Event-history models** — time-to-event with covariates.



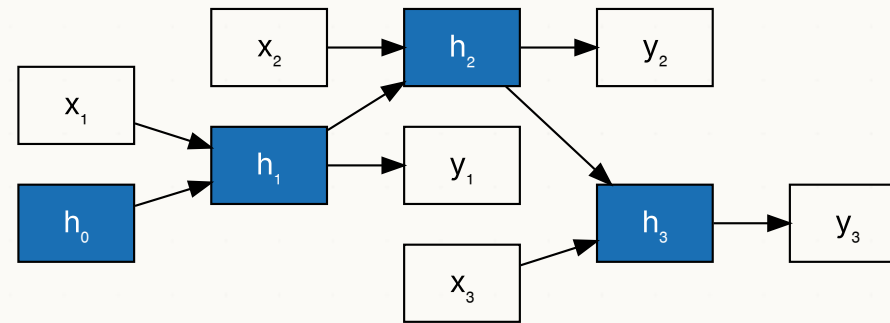
### Tip

The architecture you'll meet today applies to all of these. If your data has a natural order and prior elements inform later ones, recurrence is a candidate.

# The recurrent neural network — the idea

Process the sequence **one token at a time**. At each step, combine:

- The new input token.
- A **hidden state** carrying everything seen so far.



**Same weights** applied at every time step. The hidden state  $\mathbf{h}_t$  is the network's *running summary*.

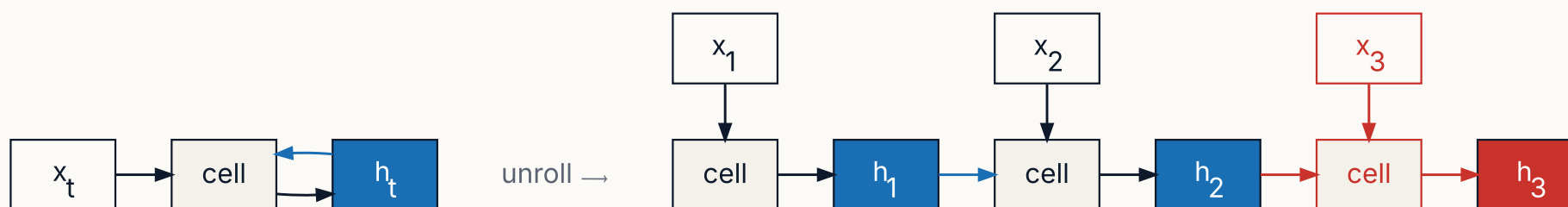


# Unrolling the recursion

THE CYCLE IS SHORTHAND — CLICK TO OPEN IT OUT

The RNN is recursive, causing a **cycle**:

- The hidden state feeds back into the cell that produced it. But Lecture 3 was clear — backprop needs an **acyclic** graph. The fix: the loop is only shorthand for a finite chain.



- Unrolling creates **one copy of the cell per token** — and the feedback becomes an input into the first, second, third copy etc.
- The three “cells” are **the same unit**
- The unrolled graph is cycle-free, so Lecture 3’s topological order and Lecture 4’s backprop apply unchanged. Outputs  $\hat{y}_t$  omitted from the picture.

# The recurrent unit, mathematically

At each time step  $t$ :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

- $\mathbf{W}_{hh}$ ,  $\mathbf{W}_{xh}$ ,  $\mathbf{W}_{hy}$  — shared across all time steps.
- $\mathbf{h}_t$  has fixed dimensionality (say, 128 or 256)
- The **only** way information from step 1 reaches step 50 is through this single hidden state (after going through 49 matmuls)



## Note

This is the **vanilla RNN** (Elman, 1990). The whole architecture is three weight matrices and a  $\tanh$ . As we'll see in Part 2, that simplicity is also its problem.

# Example forward pass

## SETUP

- Hidden dim 1.  $W_{hh} = 0.6$ ,  $W_{xh} = 1.0$ ,  $b_h = 0$ .
- $\sigma = \tanh$ . Initial  $h_0 = 0$ .
- Inputs:  $x_1 = 0.8$ ,  $x_2 = -0.4$ .

## TIME STEP 1

$$h_1 = \tanh(0.6 \cdot 0 + 1.0 \cdot 0.8) = \tanh(0.8) \approx 0.664$$

## TIME STEP 2

$$h_2 = \tanh(0.6 \cdot 0.664 + 1.0 \cdot (-0.4)) = \tanh(-0.0016) \approx -0.0016$$

## NOTICE

A negative input all but cancelled the carried-forward signal from  $x_1$ . The hidden state is only a small bit of memory, and newer inputs will overwrite older ones.

# Isn't one set of weights a limitation?

The same  $\mathbf{W}_{xh}$  processes every input token.

- The model learns *one* way to look at a token.
- The model learns *one* way to combine current input with prior history.
- No “position-specific” weights.

This is the same trick as **convolutions**

- We share parameters across positions

Parameter sharing buys us:

- **Generalisation** — what we learn “here” transfers to “there” too
- **Variable length** — no architectural change needed for longer sequences
- **Fewer parameters** — by orders of magnitude vs. an unrolled MLP

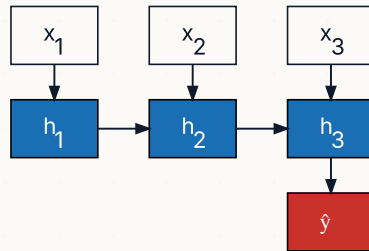
# Generation with an RNN

1. Start with  $\mathbf{h}_0 = \mathbf{0}$ .
2. Feed in a starting token  $\mathbf{x}_1$ .
3. Compute  $\mathbf{h}_1$ , then  $\hat{\mathbf{y}}_1$  — predicted next-token logits.
4. Sample next token from softmax of  $\hat{\mathbf{y}}_1$ .
5. Use it as  $\mathbf{x}_2$ . Compute  $\mathbf{h}_2$ . Sample  $\mathbf{x}_3$ .
6. Repeat as long as we want.

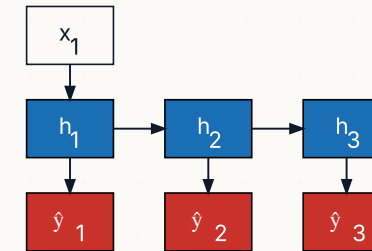
Generation is just the forward pass, repeated — same machinery as the bigram model, but with the hidden state carrying memory.

# How many inputs, how many outputs?

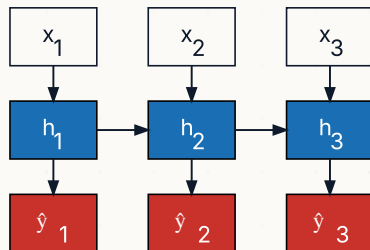
ONE RECURRENT CELL, WIRED FOUR WAYS — INPUTS ON TOP, PREDICTIONS BELOW



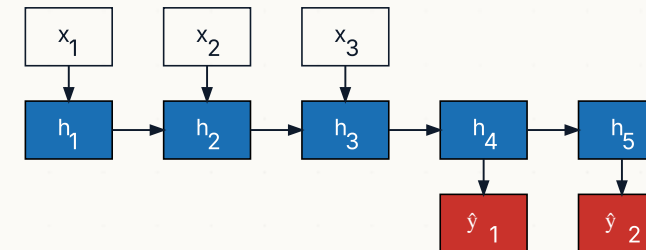
**Many-to-one** — read everything, then output one thing.  
Sentiment of a review; forecasting conflict from a sequence of events.



**One-to-many** — one input starts a sequence. Image captioning: one image in, a sentence out.



**Many-to-many, aligned** — one output per input.  
Tagging each word with its part of speech;  
predicting the next character at every position.



**Many-to-many, encoder-decoder** — read it all, then write an output of a different length. Machine translation; more at the end of today.

The cell is identical in all four panels — the same  $\mathbf{W}_{hh}$ ,  $\mathbf{W}_{xh}$ ,  $\mathbf{W}_{hy}$ . You choose where to plug in inputs and where to read off predictions.

# Bidirectional RNNs

FOR NON-GENERATIVE TASKS, YOU CAN READ BOTH DIRECTIONS

For classifying a sentence or tagging each word, the model can read **both forward and backward**:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$$

- Forward RNN reads left  $\rightarrow$  right.
- Backward RNN reads right  $\rightarrow$  left.
- Concatenate the two hidden states at each position.

## Why this isn't always usable.

- For *generation*, you can't read the future as it hasn't been produced yet.
- Bidirectional RNNs are for **labelling**, not **generating**.
- Same constraint applies to attention on Day 10: we have to apply a *causal mask* for generation, but not for classification.

# Stacked RNNs

## DEPTH IN SPACE

Layer 1's hidden states form a sequence. So we can feed them to a second RNN:

$$\mathbf{h}_t^{(2)} = \tanh\left(\mathbf{W}_{hh}^{(2)}\mathbf{h}_{t-1}^{(2)} + \mathbf{W}_{xh}^{(2)}\mathbf{h}_t^{(1)} + \mathbf{b}_h^{(2)}\right)$$

The superscript is the layer. Layer 2's "input token" at step  $t$  is layer 1's hidden state  $\mathbf{h}_t^{(1)}$ .

Two separable kinds of depth:

- **Depth in time** — how many steps the recurrence runs (stems from the data)
- **Depth in space** — how many layers are stacked (adds more parameters)

## PARAMETER COST PER EXTRA LAYER

Each layer has its own  $\mathbf{W}_{hh}$ ,  $\mathbf{W}_{xh}$ ,  $\mathbf{b}_h$ . With hidden size 128 at both layers, layer 2 adds:

$$\begin{aligned} 128^2 + 128^2 + 128 &= 16,384 + 16,384 + 128 \\ &= 32,896 \end{aligned}$$

Two or three layers was the working norm



# | Plenary · unroll an RNN by hand

# Plenary · unroll a tiny RNN

≈ 12 MIN · IN PAIRS

## A TINY RNN

Hidden dim = 1 (so  $\mathbf{h}_t$  is a single number).

Weights:  $W_{hh} = 0.5$ ,  $W_{xh} = 1.0$ ,  $b_h = 0$ . Initial  $h_0 = 0$ . Use  $\tanh$  as the activation.

(Treat the network as outputting  $h_t$  directly — skip  $\mathbf{W}_{hy}$  for this exercise.)

INPUTS AT THREE TIME STEPS  $x_1 = 1.0$ ,  $x_2 = -0.5$ ,  $x_3 = 2.0$

## COMPUTE

$h_1$ ,  $h_2$ ,  $h_3$  by hand.

## REFLECT

How much of the original  $x_1$  signal is still detectable in  $h_3$ ?

Six minutes.

# Plenary · answers

**STEP 1**  $h_1 = \tanh(0.5 \cdot 0 + 1.0 \cdot 1.0) = \tanh(1.0) \approx 0.762.$

**STEP 2**  $h_2 = \tanh(0.5 \cdot 0.762 + 1.0 \cdot (-0.5)) = \tanh(-0.119) \approx -0.119.$

**STEP 3**  $h_3 = \tanh(0.5 \cdot (-0.119) + 1.0 \cdot 2.0) = \tanh(1.94) \approx 0.959.$

## THE REFLECTION

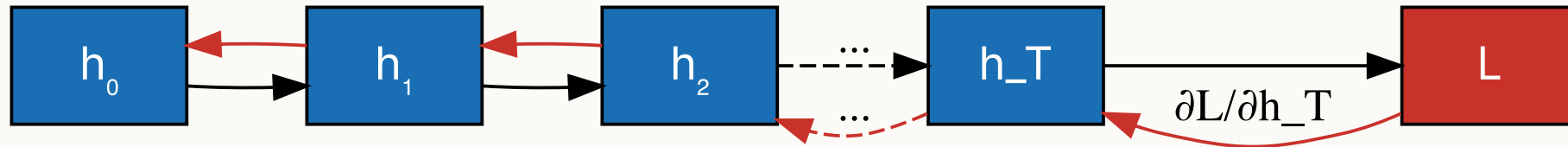
By step 3, the input  $x_3 = 2.0$  has dominated. The signal from  $x_1$  has been (a) multiplied by 0.5 at step 2, (b) squashed through  $\tanh$  at step 2, (c) multiplied by 0.5 at step 3, (d) squashed through  $\tanh$  at step 3.

That sequence of multiplications and saturations is the **vanishing gradient problem**, in miniature.

# | **Part 2 · Training & failure modes**

# Training: backpropagation through time

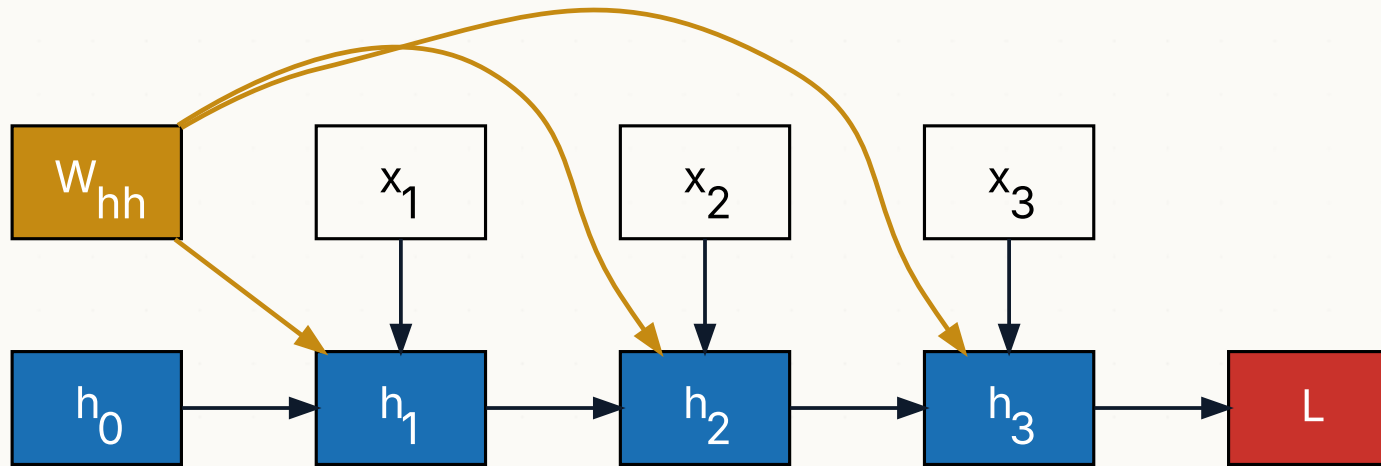
When we “unroll” the RNN over the sequence, it becomes a (very deep) **feed-forward** network where the *same* weights appear at every layer.



- **Forward pass** through the unrolled chain.
- **Backward pass** through the same chain → **backpropagation through time (BPTT)**.
- Gradients of  $\mathbf{W}_{hh}$  are **summed** contributions from *every* time step.

# Why do we sum the gradients?

ONE MATRIX, USED AT EVERY STEP — LECTURE 4'S FAN-OUT RULE, AT RNN SCALE



In computational graph terms, our RNN weights are **one node with a branch to every time step**..

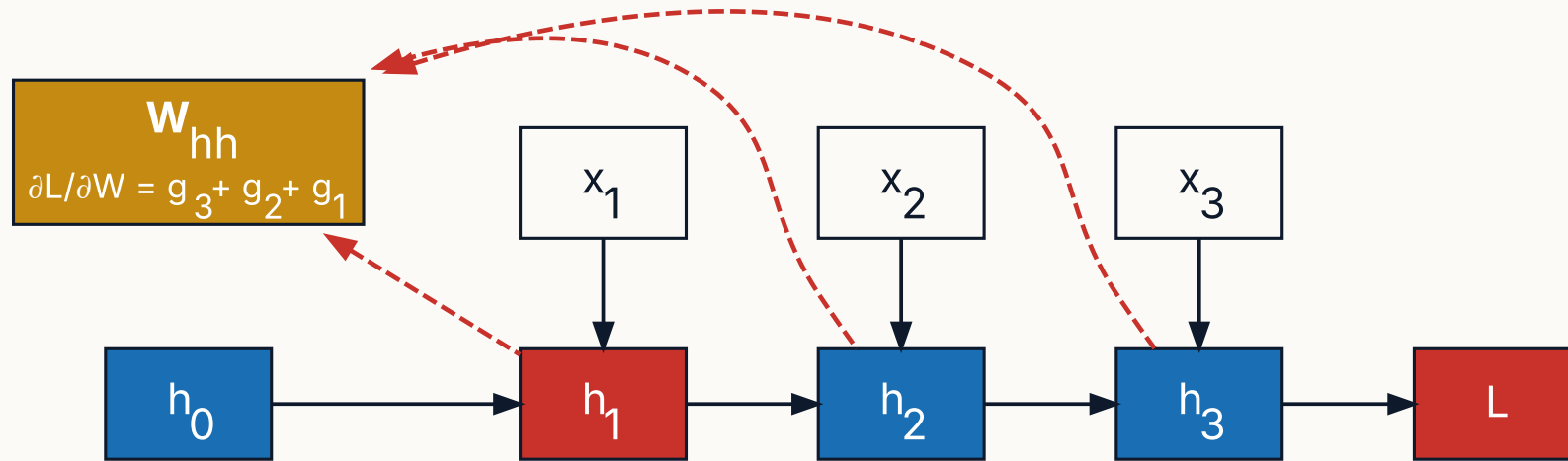
Lecture 4's rule for a value used in several places: the gradients from each use **add**. So

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \underbrace{g_1}_{\text{via } \mathbf{h}_1} + \underbrace{g_2}_{\text{via } \mathbf{h}_2} + \underbrace{g_3}_{\text{via } \mathbf{h}_3}$$

— one term per time step. That is the “summed contributions” on the last slide.

Each term travels back from  $L$  along the chain to its step — the further left the step, the more upstream nodes that influence the derivative.

THE GRAPH FROM THE LAST SLIDE, RUN BACKWARDS — ONE CLICK PER TIME STEP



- The gradients reach  $\mathbf{W}_{hh}$  **right to left**
- After all time steps are dealt with, we perform *one* update

# Teacher forcing

WHAT DO WE FEED IN WHEN THE MODEL GUESSES WRONG?

Training on the word *hello*. At each step the model predicts the next character:

| Step | Input | Target | Model's top guess |
|------|-------|--------|-------------------|
| 1    | h     | e      | e                 |
| 2    | e     | l      | x (wrong)         |
| 3    | l     | l      | l                 |
| 4    | l     | o      | o                 |

At *step 3* we feed in the **true** previous character ("*l*" from the corpus), not the "x" the model produced. This is called **teacher forcing**.

- Every position has a known target from the corpus
- The loss decomposes per step:  $L = \sum_t L_t$  — one cross-entropy per position
- A wrong guess costs loss at its own step but doesn't derail the next one



# Generation cannot use a teacher

## Training

Inputs always come from the corpus. The model reads true history at every step, mistakes or not.

## Generation

The sampled token becomes the next input. The model ingests its own output, so one wrong early token feeds every later prediction

We call this mismatch **exposure bias**.

- The model never sees its own mistakes during training, but has to accept them at generation

The alternative – training on the model's own samples – means sampling a sequence at every training step. Slow, and in practice unstable.

Transformers train with teacher forcing too, token for token

# Truncated BPTT

## TRAINING ON TEXTS TOO LONG TO UNROLL

You cannot backpropagate through a million-token book:

- The graph holds every hidden state in memory, and the gradient product runs over every step.

### Truncated BPTT:

1. Chop the text into windows of, e.g., 128 steps.
2. Run forward and backward within each window; update the weights.
3. Carry the **final hidden state** into the next window as its  $\mathbf{h}_0$ .
4. We detach this hidden state first, to prevent carrying gradients over

### Truncation preserves memory

The hidden state flows across the window boundary, so the model can carry context from window to window.

### Truncation cuts off gradient

No gradient crosses the boundary, so nothing more than 128 steps back affects the current update.

# Vanishing & exploding gradients

The gradient of the loss w.r.t.  $\mathbf{h}_1$ :

$$\frac{\partial L}{\partial \mathbf{h}_1} = \frac{\partial L}{\partial \mathbf{h}_T} \cdot \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_{T-1}} \cdots \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}$$

Each factor  $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$  is the matrix of partial derivatives of  $\mathbf{h}_t$  with respect to  $\mathbf{h}_{t-1}$ .

With  $T$  **multiplications** of similar matrices, and magnitudes generally  $< 1$ , the product shrinks.

- And if  $> 1$ , it explodes.

## Vanishing

Early steps stop learning. The model effectively has **no long-range memory**.

This is the harder problem.

## Exploding

Training diverges, but can be fixed in practice with **gradient clipping**.

# Vanilla RNNs struggle past ~50 tokens

Recall  $\tanh'$  peaks at 1 and falls off fast. For  $|z| > 5$ ,  $\tanh'(z) < 0.001$ .

After 50 multiplications by a matrix whose largest singular value is, e.g., 0.5:

$$0.5^{50} \approx 8.9 \times 10^{-16}$$

About  $10^{-15}$  of the signal arrives.

- So whatever happened 50 steps ago has no measurable influence on the update.



**Often, real RNNs degrade well before 50 tokens**

RNNs handle sequences of 5–20 tokens reasonably well, but any larger and you need some variant.

# | **Activity · find the vanishing gradient**

# Activity · find the vanishing gradient

≈ 8 MIN · IN PAIRS

A vanilla RNN with hidden dim 1,  $W_{hh} = 0.3$ ,  $W_{xh} = 1.0$ ,  $b_h = 0$ ,  $\tanh$  activation.

Inputs:  $x_1, x_2, \dots, x_{10}$  — all equal to 1.0.

COMPUTE (APPROXIMATELY)

1.  $h_1, h_2, h_5, h_{10}$ .

2. Estimate  $\frac{\partial h_{10}}{\partial h_5}$  — what's the magnitude?

3. **Interpretation:** if I change  $x_5$  by a small amount, how much does  $h_{10}$  change?

Six minutes.

# Activity · answer

## TRAJECTORY OF $h_t$

$h_1 = \tanh(1.0) \approx 0.762$ . Then each step:  $h_{t+1} = \tanh(0.3h_t + 1.0)$ .

Numerically:  $h_2 = \tanh(1.2285) \approx 0.842$ ,  $h_3 = \tanh(1.2527) \approx 0.849$ , ..., converges to a fixed point  $\approx 0.850$ .

## GRADIENT

$$\frac{\partial h_{t+1}}{\partial h_t} = (1 - h_{t+1}^2) \cdot 0.3.$$

At convergence ( $h \approx 0.850$ ):  $(1 - 0.850^2)(0.3) = (0.2775)(0.3) \approx 0.083$ .

$$\text{So } \frac{\partial h_{10}}{\partial h_5} \approx 0.083^5 \approx 3.9 \times 10^{-6}.$$

## INTERPRETATION

Changing  $x_5$  moves  $h_{10}$  by **about a millionth** of the change you made. The signal vanished in 5 steps.

Backprop can't tell the network that  $x_5$  matters for predicting  $\hat{y}_{10}$ . Training stalls on long-range dependencies.

# Exploding gradients

THIS IS THE SAME MECHANISM BUT RUN THE OTHER WAY

If we drop the  $\tanh$  (a simplification), the **linear** RNN:

$$h_t = 1.5 h_{t-1} + x_t$$

has a backward step that now multiplies the gradient by 1.5:

$$\frac{\partial h_{10}}{\partial h_5} = 1.5^5 \approx 7.6 \qquad 1.5^{50} \approx 6.4 \times 10^8$$

Fifty steps turn 1.5 into 640 million!

With the  $\tanh$  back in, each factor is  $W_{hh} \tanh'(\cdot)$  — messier, but these failure modes remain.



# Gradient clipping

FOR EXPLODING GRADIENTS WE CAN BE PRAGMATIC

**Algorithm:** after backprop, compute the norm of all gradients. If it exceeds a threshold (e.g. 1.0 or 5.0), **rescale** all gradients to that threshold.

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(), max_norm=1.0
)
```

- Can be added to **every** RNN training loop in practice.
- Stops the loss from going NaN as  $L \rightarrow \infty$

For exploding, clipping is enough.

- An example of “if the maths is unstable, just truncate” that recurs across deep learning



**Doesn't solve vanishing gradients**

To “clip” a number close to 0 would only worsen the problem!

# Gated multiplication

CLIPPING HANDLED EXPLODING. FOR VANISHING, THE FIX IS A MULTIPLY THE NETWORK CONTROLS

A **gate** is a vector with entries between 0 and 1, produced by a small sigmoid layer with its own learned weights:

$$\mathbf{g} = \sigma(\mathbf{W}_g[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_g)$$

$[\mathbf{h}_{t-1}, \mathbf{x}_t]$  stacks the two vectors into one long input — notation we'll reuse for the LSTM.

Applying a gate is **element-wise multiplication** (the *Hadamard product*), written  $\odot$ :

- Multiply matching entries of *two equal-shaped* vectors

## WORKED EXAMPLE

$$\begin{bmatrix} 1.0 \\ 0.5 \\ 0.0 \end{bmatrix} \odot \begin{bmatrix} 4.0 \\ -2.0 \\ 5.0 \end{bmatrix} = \begin{bmatrix} 4.0 \\ -1.0 \\ 0.0 \end{bmatrix}$$

Maintains the same shape.

## WHY A GATE HELPS

$\mathbf{W}_g$  is learned and  $\mathbf{g}$  is recomputed at every step, so the network reads  $\mathbf{h}_{t-1}$  and  $\mathbf{x}_t$  and decides what passes.

The  $\tanh$  chain multiplied by  $\approx 0.083$  per step whether the signal mattered or not. A gate can learn to open fully and multiply by  $\$1.0$ .

$\sigma$  never reaches 0 or 1 exactly and instead sits at values like 0.002 or 0.998

# Gated RNNs

## LSTM (Hochreiter & Schmidhuber, 1997)

A **cell state**  $\mathbf{c}_t$  that can carry information across many steps with only **gated multiplications**

- Enabling gradients to survive much longer

Three gates:

- **Forget:** drop bits of  $\mathbf{c}_{t-1}$ .
- **Input:** add bits to  $\mathbf{c}_t$ .
- **Output:** expose part of  $\mathbf{c}_t$  as  $\mathbf{h}_t$ .

LSTMs and GRUs were the state of the art for sequence tasks from ~2014 to 2017. Then came **transformers** — the story at the end of today, and all of Monday.

## GRU

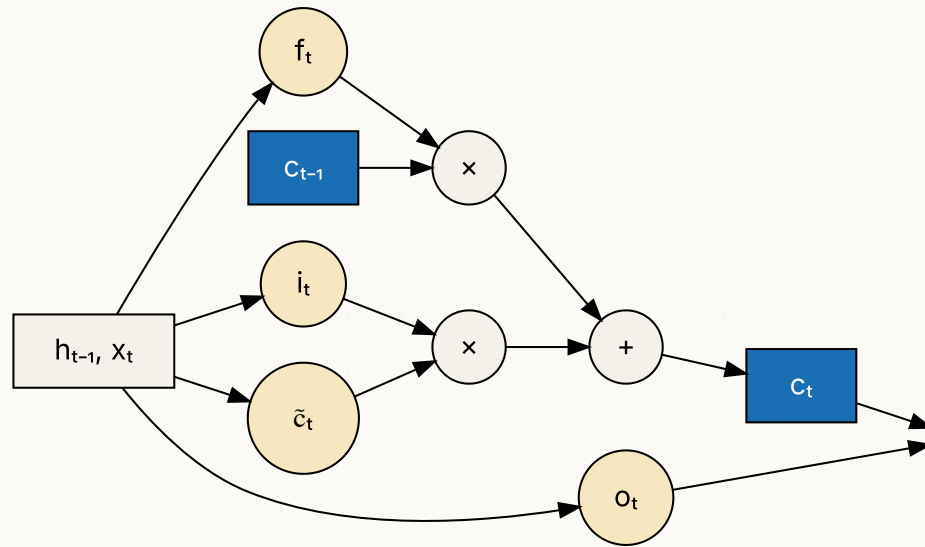
A simpler version (Cho et al. 2014)

- Fewer gates than LSTM
- Comparable performance on most tasks
- Cheaper

This is the default RNN variant in many libraries

# Inside an LSTM cell

THE DIAGRAM FIRST — THE EQUATIONS COME NEXT SLIDE



## THREE GATES + A CELL STATE

- **Forget gate  $f_t$**  — how much of the previous cell state  $c_{t-1}$  to keep.
- **Input gate  $i_t$**  — how much of the new candidate  $\tilde{c}_t$  to write.
- **Output gate  $o_t$**  — how much of the updated  $c_t$  to expose as the hidden state.

The straight path along the top —  $c_{t-1} \rightarrow c_t$  — is the **memory highway**: information rides it with only a multiply and an add.

# The LSTM, in equations

SAME PICTURE, WRITTEN OUT. EACH GATE IS ITS OWN SIGMOID LAYER;  $\odot$  IS THE ELEMENT-WISE MULTIPLY FROM THE GATING SLIDE.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

The three gates are **sigmoids**, so each entry sits in  $[0, 1]$  — a soft “how much to let through”.

## WORKED FORGET-GATE NUMBER

A forget gate of  $f_t = 0.9$  keeps 90% of the old cell value: if  $c_{t-1} = 2.0$ , the forget step alone passes  $0.9 \times 2.0 = 1.8$  forward. A gate of 0.1 would pass only  $0.1 \times 2.0 = 0.2$ .

The cell update is **additive** — gradients flow along  $\mathbf{c}$  without shrinking.

# One LSTM step by hand

ONE DIMENSION OF EVERYTHING, SO EACH QUANTITY IS A SINGLE NUMBER

## SETUP

- $h_0 = 0, c_0 = 0$ , input  $x_1 = 1$ .
- Each weight matrix is a pair  
[ weight on  $h_{t-1}$ , weight on  $x_t$  ]:

$$W_f = [0, 0.5]$$

$$W_i = W_o = W_c = [0, 1]$$

$$b_f = b_i = b_o = b_c = 0$$

## THE THREE GATES

$$f_1 = \sigma(0 \times 0 + 0.5 \times 1 + 0) = \sigma(0.5) \approx 0.62$$

$$i_1 = \sigma(0 \times 0 + 1 \times 1 + 0) = \sigma(1) \approx 0.73$$

$$o_1 = \sigma(0 \times 0 + 1 \times 1 + 0) = \sigma(1) \approx 0.73$$

All three numbers are between 0 and 1.

# One LSTM step by hand — memory and output

IN ONE DIMENSION,  $\odot$  IS JUST MULTIPLICATION

CANDIDATE

$$\tilde{c}_1 = \tanh(0 \times 0 + 1 \times 1 + 0) = \tanh(1) \approx 0.76$$

MEMORY UPDATE

$$c_1 = f_1 c_0 + i_1 \tilde{c}_1 \approx (0.62 \times 0) + (0.73 \times 0.76) \approx 0.56$$

OUTPUT

$$h_1 = o_1 \tanh(c_1) \approx 0.73 \times \tanh(0.56) \approx 0.73 \times 0.51 \approx 0.37$$

# One LSTM step by hand — reading the numbers

- $f_1 \approx 0.62$  — *keep 62% of the old memory.* ( $c_0 = 0$  here, so there was nothing to keep yet.)
- $i_1 \approx 0.73$  — *write 73% of the new candidate into memory.*
- $o_1 \approx 0.73$  — *expose 73% of the (squashed) memory as the hidden state.*

Each gate is a ***learned*** dial between 0 and 1.

- In a trained model the forget gate might sit near 1 while a name is being spelled out...
- ... then drop towards 0 at a full stop



# Inside a GRU cell — the simpler cousin

CHO ET AL. (2014) — TWO GATES, NO SEPARATE CELL STATE

## TWO GATES

- **Update gate**  $\mathbf{z}_t$  — how much of the new candidate  $\tilde{\mathbf{h}}_t$  to write in;  $(1 - \mathbf{z}_t)$  keeps the old state.
- **Reset gate**  $\mathbf{r}_t$  — how much of the previous hidden state to use when computing the candidate.

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

**Pros:** roughly 25% fewer parameters than an LSTM.  
Faster to train. Comparable performance on most tasks.

**Cons:** slightly less expressive for very long sequences.  
Hard to tell apart from LSTMs in benchmarks.



### Precise gating doesn't matter

LSTM and GRU achieve similar performance with different gate structures. What really matters is having *some* additive path for gradients to flow through long sequences.

# What the cells learn

KARPATHY, [THE UNREASONABLE EFFECTIVENESS OF RECURRENT NEURAL NETWORKS](#) (2015)

Karpathy:

1. Trained character-level LSTMs on prose and on source code
2. Plotted the value of **individual cells** as the model read.

- One cell tracked whether the text was **inside a quotation** – one value between quote marks, another outside.
- One cell tracked **line length** – resetting at each newline.
- One cell tracked **bracket depth** in code.

*Nobody specified these!* Next-character prediction alone caused these features to emerge because they help.

# The hidden state is a learned summary

## The general point

$h_t$  is a **distributed summary of what matters so far** – learned from the prediction objective.

- Nobody wrote a quote detector. The model built one because quotes change what comes next.
- A few cells are readable. Most are not — the summary is spread across all of them at once.

At only 128 hidden units, you can plot every cell and hunt for the readable ones (as Karpathy did).

At millions of units, however, single-cell inspection stops working

- Understanding what a model represents becomes a research field of its own
- Called **interpretability** or “explainable AI” (xAI)
- This problem returns with transformers

# Why RNNs lost

## The constraint

To compute  $\mathbf{h}_t$ , you need  $\mathbf{h}_{t-1}$ .

- **No parallelism over time.**
- Training a length- $T$  sequence takes time proportional to  $T$ .
- GPUs love parallelism

## Transformers

Process every token **in parallel** during training.

- Massive throughput
- Scale to ~~billions~~ trillions of tokens
- Same architecture at every size

Vaswani et al. ([2017](#)) — *"Attention is all you need"*.

**The transformer paper is short and readable**, so I'd recommend reading it

- We'll go through Figure 1 line-by-line on Monday morning

## **| Part 3 · Where RNNs sit today**

# Encoder-decoder seq2seq — the pre-transformer winner

SEQ2SEQ = *SEQUENCE-TO-SEQUENCE: THE ARCHITECTURE THAT POWERED MACHINE TRANSLATION 2014–2017*

## Encoder

Reads the input sentence with an RNN. The **final hidden state**  $\mathbf{h}_T$  becomes a single fixed-size summary.

## DECODER

A second RNN, initialised from  $\mathbf{h}_T$ . Generates the output one token at a time.

**The bottleneck.** All information about the input has to squeeze through one vector  $\mathbf{h}_T$ . For long sentences this breaks down — as the next slide makes concrete.

# A foreshadow of attention

## THE SEQ2SEQ BOTTLENECK, IN ONE EXAMPLE

Translating a 30-word sentence with vanilla seq2seq:

1. Encoder reads all 30 words → final hidden state  $\mathbf{h}_{30}$ .
2. Decoder starts from  $\mathbf{h}_{30}$ .
3. By the time the decoder is on word 5, it has *forgotten* the start of the input.

The information from “the cat” at the beginning has to travel through 30 RNN updates *and then* 5 decoder updates

- The same problem: 35 multiplications by partial derivative matrices with magnitude  $< 1$



### Tip

Bahdanau et al. (2014) fixed this by letting the decoder *look back* at *all* the encoder’s hidden states — a weighted average, with weights the model learns to set. They called it **attention**.

It worked so well that, by 2017, someone asked: do we even need the recurrent layers?

# Still in use

---

Despite losing the headline race, RNNs (especially LSTMs / GRUs) are still chosen for:

- **Streaming applications** — real-time speech recognition, online time-series control.
- **Small-data settings** — LSTMs need less training data than transformers.
- **Time-series forecasting** — where the autoregressive structure is a *feature*.
- **Embedded systems** — RNNs run on tiny chips.

## Social-science examples

- Forecasting **conflict events** as a sequence over time ([Hegre et al., 2022](#)).
- **Roll-call vote prediction** — modelling votes within a legislative session.
- **Event-history models** as DL — survival analysis with sequential covariates.
- **Sentiment over conversations** — speaker-to-speaker.



# RNNs for survival analysis

## A SOCIAL-SCIENCE APPLICATION OF SEQ2ONE RNNs

Standard survival analysis (Cox PH, Kaplan-Meier) assumes:

- Proportional hazards.
- Fixed covariates (or simple time-varying ones).
- Linear-in-covariates risk.

**Recurrent survival models** relax all three:

- The hidden state  $\mathbf{h}_t$  summarises the full event history up to time  $t$ .
- Output is the hazard rate at each time step.
- Non-linear interactions emerge from the network.

Applications:

- **Patient trajectories** (re-admission, mortality).
- **Customer churn** with rich behavioural sequences.
- **Time-to-conflict-onset** in international relations.
- **Firm bankruptcy** with quarterly financials.

A drop-in upgrade when you have rich sequences and enough data to make the network worth its complexity.

# A worked social-science example

## PREDICTING PROTEST EVENTS FROM A STREAM OF NEWS HEADLINES

- **Input:** a sequence of daily news headlines for a country.
- **Output:** probability of a protest in each subsequent day.
- The model needs **memory** of recent escalating language.
- An LSTM (or GRU) is a sensible architecture.

Even with transformers available, an LSTM-based forecaster is often:

- Easier to train on small national news corpora.
- Easier to deploy in real time on rolling data.
- Easier to interpret per-timestep contribution.



### Choosing between architectures

Ask not “what is the biggest model?” but “what is the right model for the data I have?”

- Transformers work best when you have lots of data and lots of compute.
- RNNs often work better when you have less data and/or compute

# Applications beyond text

WHERE RNNs/LSTMS ARE STILL LOAD-BEARING

## Speech

- End-to-end speech recognition (CTC-based RNNs from ~2014).
- Live captioning systems.
- Many production ASR pipelines retain RNNs.

## Music

- MIDI generation, ad-hoc accompaniment.
- OpenAI's MuseNet had transformer + RNN components.

## Time series

- Forecasting (sales, demand, finance).
- Anomaly detection (server logs, IoT sensors).
- Often beats classical models like ARIMA on long series.

## Bioinformatics

- DNA / protein sequence modelling (pre-AlphaFold).
- Variant-effect prediction.



### Tip

The common thread: moderate sequence lengths, streaming data, modest compute requirements. These are conditions where transformers' parallel training will buy little.

# Lessons RNNs taught us

---

CARRY INTO THE REST OF THE COURSE

1. **Parameter sharing** across position (also true of CNNs, attention).
2. **Hidden state** as compressed running summary (returns in transformers' KV cache).
3. **The vanishing-gradient problem** (returns in deep transformers — fixed with residuals + LayerNorm).
4. **Gating** as a way to control information flow (returns in transformer MoE, mixture-of-experts routing).
5. **Encoder-decoder** as a template (still the shape of translation, summarisation).

Even though recurrence is now “out of date”, its constituent parts remain very useful!

# What we covered today

---

## TAKE STOCK

You can:

1. Explain why feed-forward networks don't naturally handle variable-length sequences.
2. Describe the vanilla RNN — recurrence relation, hidden state, parameter sharing.
3. Run one by hand for a small number of steps.
4. State the vanishing/exploding gradient problem and why it's worse for RNNs.
5. Name LSTM and GRU as the gated fixes, and explain why transformers ultimately replaced them.
6. Cite at least one social-science setting where an RNN is still the right tool.

# | **Lab brief • Monday**

# Monday's lab — 90 minutes

---

## GOALS

Take yesterday's character-level bigram model and:

1. Replace the embedding-as-logits with a proper **RNN cell** ( `nn.RNN` or `nn.GRU` ).
2. Carry the hidden state across each character of a sequence.
3. Predict the next character at every position.
4. Train for ~5k steps on the same corpus.
5. Generate a 500-character sample.

## WHAT YOU SHOULD SEE

The samples will be noticeably *less* terrible. You'll see real(ish) words start to appear and sensible whitespace.

Stretch goal: train with sequence length 64, 128, 256 and plot the loss for each. Does more context keep paying?

# See you on Monday

LECTURE 10 — ATTENTION & TRANSFORMERS



## Tip

**Over the weekend, if you want a preview:** read the *abstract* of Vaswani et al. (2017). Stop there. We'll do the rest together on Monday.

Hegre, H., Akbari, F., Croicu, M., Dale, J., Gåsste, T., Jansen, R., Landsverk, P., Leis, M., Lindqvist-McGowan, A., Mueller, H., et al. (2022). *Forecasting fatalities*.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9, 1735–1780.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.